

Práctica Integradora Final

Programación en C — Todos los Temas

Soluciones Completas y Comentadas

Materia: Programación en C

Parte 1: Completar código (Casos 1 y 2)

Parte 2: Modificar código (Casos 3 y 4)

Índice

1. PARTE 1: Completar Código	2
1.1. Caso 1: Sistema de Gestión de Biblioteca	2
1.2. Caso 2: Análisis de Temperaturas Mensuales	5
2. PARTE 2: Modificar Código	8
2.1. Caso 3: Corregir Sistema de Inventario	8
2.2. Caso 4: Optimizar Procesamiento de Datos	11

1. PARTE 1: Completar Código

1.1. Caso 1: Sistema de Gestión de Biblioteca

Contexto: Una biblioteca necesita un sistema para gestionar préstamos de libros. Se tienen 5 libros en el inventario y se requiere procesar múltiples archivos de préstamos. El programa lee `prestamo_1.txt`, `prestamo_2.txt` y `prestamo_3.txt`, marca los libros como no disponibles y muestra el porcentaje prestado.

Líneas completadas — explicación

Código completado	Explicación
<code>sprintf(nombre, "prestamo_%d.txt", i)</code>	Construye el nombre del archivo usando el número de iteración
<code>archivo = fopen(nombre, r")</code>	Abre el archivo en modo lectura (<code>r</code>)
<code>fscanf(archivo, "%d", &codigo_prestamo)</code>	Lee el entero (código del libro) del archivo
<code>biblioteca[j].codigo == codigo_prestamo</code>	Compara el código del libro con el prestado
<code>biblioteca[i].disponible == 1</code>	Filtra solo los libros disponibles para mostrarlos
<code>(5 - contador_disponibles) * 100.0 / 5</code>	Calcula el porcentaje de libros prestados

Código solución completo

```

1 #include <stdio.h>
2 #include <string.h>
3
4 typedef struct {
5     int    codigo;
6     char   titulo[50];
7     char   autor[30];
8     int    disponible;
9 } Libro;
10
11 int main() {
12     Libro biblioteca[5] = {
13         {101, "Cien Anos de Soledad", "Garcia Marquez", 1},
14         {102, "Don Quijote",          "Cervantes",      1},
15         {103, "El Principito",        "Saint-Exupery", 1},
16         {104, "1984",                 "Orwell",         1},
17         {105, "Rayuela",               "Cortazar",       1}

```

```
18     };
19
20     FILE *archivo;
21     char nombre[30];
22     int codigo_prestamo;
23
24     for (int i = 1; i <= 3; i++) {
25         /* Construir nombre del archivo */
26         sprintf(nombre, "prestamo_%d.txt", i);
27         /* Abrir en modo lectura */
28         archivo = fopen(nombre, "r");
29
30         if (archivo != NULL) {
31             /* Leer el codigo del libro prestado */
32             fscanf(archivo, "%d", &codigo_prestamo);
33
34             /* Buscar y marcar como no disponible */
35             for (int j = 0; j < 5; j++) {
36                 if (biblioteca[j].codigo == codigo_prestamo) {
37                     biblioteca[j].disponible = 0;
38                 }
39             }
40             fclose(archivo);
41         }
42     }
43
44     printf("LIBROS DISPONIBLES:\n");
45     printf("%-5s %-30s %-20s\n", "Cod", "Titulo", "Autor");
46     printf("
47     -----\n
48     ");
49
50     int contador_disponibles = 0;
51     for (int i = 0; i < 5; i++) {
52         /* Solo mostrar libros con disponible == 1 */
53         if (biblioteca[i].disponible == 1) {
54             printf("%-5d %-30s %-20s\n",
55                 biblioteca[i].codigo,
56                 biblioteca[i].titulo,
57                 biblioteca[i].autor);
58             contador_disponibles++;
59         }
60     }
61
62     /* Porcentaje prestado = libros no disponibles / total * 100
63     */
64     float porcentaje = (5 - contador_disponibles) * 100.0 / 5;
65     printf("\nLibros prestados: %.1f%%\n", porcentaje);
66
67     return 0;
68 }
```

Listing 1: Caso 1 — Sistema de Gestión de Biblioteca

Archivos de entrada requeridos

Archivo	Contenido
prestamo_1.txt	102
prestamo_2.txt	104
prestamo_3.txt	101

Salida esperada

LIBROS DISPONIBLES:

Cod	Titulo	Autor

103	El Principito	Saint-Exupery
105	Rayuela	Cortazar

Libros prestados: 60.0%

1.2. Caso 2: Análisis de Temperaturas Mensuales

Contexto: Se tienen registros de temperatura diaria de 30 días. El programa lee los datos desde `temperaturas.txt`, calcula estadísticas (promedio, máximo, mínimo, días calurosos) y guarda un reporte en `reporte.txt`. Se utiliza una arquitectura modular con funciones separadas.

Líneas completadas — explicación

Código completado	Explicación
<code>suma += temps[i]</code>	Acumula todos los valores del arreglo
<code>return suma / n</code>	Divide la suma entre la cantidad de elementos
<code>if (temps[i] > max)</code>	Compara el elemento actual con el máximo encontrado
<code>(función encontrar_minimo)</code>	Misma lógica que máximo pero con operador <
<code>count++</code>	Incrementa el contador de días calurosos
<code>calcular_promedio(temps, total)</code>	Llama a la función con el arreglo y cantidad de días
<code>encontrar_maximo(temps, total)</code>	Llama a la función para obtener la temperatura más alta
<code>encontrar_minimo(temps, total)</code>	Llama a la función para obtener la temperatura más baja
<code>contar_calurosos(temps, total)</code>	Llama a la función que cuenta días sobre 25°C
<code>fopen(reporte.txt, "w")</code>	Abre el archivo de salida en modo escritura
<code>fclose(salida)</code>	Cierra el archivo para guardar los datos correctamente

Código solución completo

```

1 #include <stdio.h>
2
3 /* Calcula el promedio de n elementos */
4 float calcular_promedio(float temps[], int n) {
5     float suma = 0;
6     for (int i = 0; i < n; i++) {
7         suma += temps[i];          /* Acumular suma */
8     }
9     return suma / n;              /* Dividir por cantidad */
10 }
```

```
11
12 /* Retorna el valor maximo del arreglo */
13 float encontrar_maximo(float temps[], int n) {
14     float max = temps[0];
15     for (int i = 1; i < n; i++) {
16         if (temps[i] > max) { /* Si es mayor, actualizar */
17             max = temps[i];
18         }
19     }
20     return max;
21 }
22
23 /* Retorna el valor minimo del arreglo */
24 float encontrar_minimo(float temps[], int n) {
25     float min = temps[0];          /* Iniciar con el primer
26         elemento */
27     for (int i = 1; i < n; i++) {
28         if (temps[i] < min) { /* Si es menor, actualizar */
29             min = temps[i];
30         }
31     }
32     return min;
33 }
34
35 /* Cuenta dias cuya temperatura supera los 25 grados */
36 int contar_calurosos(float temps[], int n) {
37     int count = 0;
38     for (int i = 0; i < n; i++) {
39         if (temps[i] > 25.0) {
40             count++;          /* Incrementar contador */
41         }
42     }
43     return count;
44 }
45
46 int main() {
47     float temperaturas[30];
48     int total = 0;
49
50     FILE *entrada = fopen("temperaturas.txt", "r");
51     while (fscanf(entrada, "%f", &temperaturas[total]) == 1) {
52         total++;
53     }
54     fclose(entrada);
55
56     /* Calcular estadisticas usando las funciones */
57     float promedio = calcular_promedio(temperaturas, total);
58     float max_temp = encontrar_maximo(temperaturas, total);
59     float min_temp = encontrar_minimo(temperaturas, total);
60     int dias_calurosos = contar_calurosos(temperaturas, total);
```

```
61  /* Guardar reporte en archivo de salida */
62  FILE *salida = fopen("reporte.txt", "w");
63  fprintf(salida, "REPORTE DE TEMPERATURAS\n");
64  fprintf(salida, "=====\n\n");
65  fprintf(salida, "Total de dias: %d\n",          total);
66  fprintf(salida, "Temperatura promedio: %.2fC\n", promedio);
67  fprintf(salida, "Temperatura maxima: %.2fC\n", max_temp);
68  fprintf(salida, "Temperatura minima: %.2fC\n", min_temp);
69  fprintf(salida, "Dias calurosos (>25C): %d\n", dias_calurosos);
70  fclose(salida);                               /* Cerrar archivo de salida */
71
72  printf("Reporte generado exitosamente\n");
73
74  return 0;
75 }
```

Listing 2: Caso 2 — Análisis de Temperaturas Mensuales

Salida esperada en reporte.txt

```
REPORTE DE TEMPERATURAS
=====

Total de dias: 30
Temperatura promedio: 25.77C
Temperatura maxima: 29.10C
Temperatura minima: 22.50C
Dias calurosos (>25C): 17
```

2. PARTE 2: Modificar Código

2.1. Caso 3: Corregir Sistema de Inventario

Problema: El código original tiene 4 errores que impiden su correcta ejecución. A continuación se identifican y corrigen cada uno.

Los 4 errores identificados

Error 1. Condición del bucle — línea 20

```
for(int i = 0; i <= 3; i++)
```

El arreglo tiene 3 elementos (índices 0, 1, 2). Con `i <= 3` se accede al índice 3 que **no existe**, produciendo un desbordamiento de arreglo y resultados incorrectos.

Corrección: `i < 3`

Error 2. Argumentos de strcpy invertidos — línea 28

```
strcpy("Monitor", nuevo.nombre)
```

La función `strcpy` recibe **primero el destino, luego la fuente**. Escribir en un literal de cadena produce comportamiento indefinido.

Corrección: `strcpy(nuevo.nombre, "Monitor")`

Error 3. Falta cerrar el archivo

Después del `fprintf` no hay `fclose()`. Sin cerrar el archivo, los datos pueden no guardarse correctamente (pérdida de buffer).

Corrección: agregar `fclose(archivo);`

Error 4. Mismo error que Error 1

El enunciado lista el error de condición del bucle dos veces (`i<=3` debe ser `i<3`), ya corregido en el punto 1.

Código corregido completo

```
1 #include <stdio.h>
2 #include <string.h>
3
4 typedef struct {
5     int    codigo;
6     char   nombre[30];
7     float  precio;
8     int    stock;
9 } Producto;
10
11 int main() {
12     Producto inventario[3] = {
13         {1, "Laptop", 2500.0, 10},
14         {2, "Mouse", 35.0, 50},
```

```

15     {3, "Teclado", 85.0, 30}
16 };
17
18 float valor_total = 0;
19
20 /* CORRECCION 1: i < 3 (no i <= 3)      evita acceso fuera
    del arreglo */
21 for (int i = 0; i < 3; i++) {
22     valor_total += inventario[i].precio * inventario[i].
        stock;
23 }
24
25 printf("Valor total del inventario: S/ %.2f\n", valor_total);
26
27 Producto nuevo;
28 nuevo.codigo = 4;
29
30 /* CORRECCION 2: destino primero, fuente segundo */
31 strcpy(nuevo.nombre, "Monitor");
32
33 nuevo.precio = 450.0;
34 nuevo.stock = 15;
35
36 FILE *archivo = fopen("nuevo_producto.txt", "w");
37 fprintf(archivo, "%d,%s,%.2f,%d\n",
38         nuevo.codigo, nuevo.nombre,
39         nuevo.precio, nuevo.stock);
40
41 /* CORRECCION 3: cerrar el archivo */
42 fclose(archivo);
43
44 printf("Producto agregado al archivo\n");
45
46 return 0;
47 }

```

Listing 3: Caso 3 — Inventario con los 4 errores corregidos

Verificación del valor total

Producto	Precio	Stock	Subtotal
Laptop	S/ 2500.00	10	S/ 25000.00
Mouse	S/ 35.00	50	S/ 1750.00
Teclado	S/ 85.00	30	S/ 2550.00
TOTAL			S/ 30425.00

Valor total del inventario: S/30425.00
Producto agregado al archivo

2.2. Caso 4: Optimizar Procesamiento de Datos

Problema: El código original repite el mismo bloque de suma y promedio tres veces. La tarea es crear una función `calcular_promedio()` para eliminar el código duplicado y hacer el programa más limpio y mantenible.

Comparación: antes y después

Antes (código repetido)	Después (refactorizado)
<pre>float suma_enero = 0; for(int i=0; i<5; i++) suma_enero += ventas_enero[i]; float prom = suma_enero / 5; (<i>repetido 3 veces = 12 líneas</i>)</pre>	<pre>calcular_promedio(ventas_enero, 5) calcular_promedio(ventas_febrero, 5) calcular_promedio(ventas_marzo, 5) (<i>3 llamadas = 3 líneas</i>)</pre>

Código refactorizado completo

```

1  #include <stdio.h>
2
3  /*
4   * Calcula el promedio de un arreglo de floats.
5   * Parametros:
6   *   ventas[] -> arreglo con los valores de ventas
7   *   n        -> cantidad de elementos en el arreglo
8   * Retorna: el promedio como float
9   */
10 float calcular_promedio(float ventas[], int n) {
11     float suma = 0;
12     for (int i = 0; i < n; i++) {
13         suma += ventas[i];
14     }
15     return suma / n;
16 }
17
18 int main() {
19     float ventas_enero[5]    = {1200, 1500, 1350, 1600, 1450};
20     float ventas_febrero[5] = {1400, 1650, 1500, 1700, 1550};
21     float ventas_marzo[5]   = {1300, 1550, 1450, 1650, 1500};
22
23     /* Usar la funcion tres veces en lugar de repetir el bucle
24      */
25     printf("Promedio Enero:    %.2f\n", calcular_promedio(
26         ventas_enero,    5));

```

```
25     printf("Promedio Febrero: %.2f\n", calcular_promedio(  
        ventas_febrero, 5));  
26     printf("Promedio Marzo: %.2f\n", calcular_promedio(  
        ventas_marzo, 5));  
27  
28     return 0;  
29 }
```

Listing 4: Caso 4 — Ventas refactorizadas con función reutilizable

Verificación de resultados

Mes	V1	V2	V3	V4	V5	Promedio
Enero	1200	1500	1350	1600	1450	$\frac{7100}{5} = \mathbf{1420,00}$
Febrero	1400	1650	1500	1700	1550	$\frac{7800}{5} = \mathbf{1560,00}$
Marzo	1300	1550	1450	1650	1500	$\frac{7450}{5} = \mathbf{1490,00}$

```
Promedio Enero: 1420.00  
Promedio Febrero: 1560.00  
Promedio Marzo: 1490.00
```